Master Equipment List (MEL) Management System

Enterprise Architecture & Technical Specification

1. Executive Summary & System Objectives

The Master Equipment List (MEL) Management System is an enterprise-grade BIM integration platform designed to establish a single, bi-directionally synchronized source of truth for all mechanical, electrical, plumbing, and process equipment across complex capital project lifecycles. By bridging multi-model Autodesk Construction Cloud (ACC) environments with external project controls, procurement systems, and field operations, the platform mitigates the data decay, manual re-entry errors, and sync-lock friction inherent in legacy workflows.

Core Business & Engineering Objectives

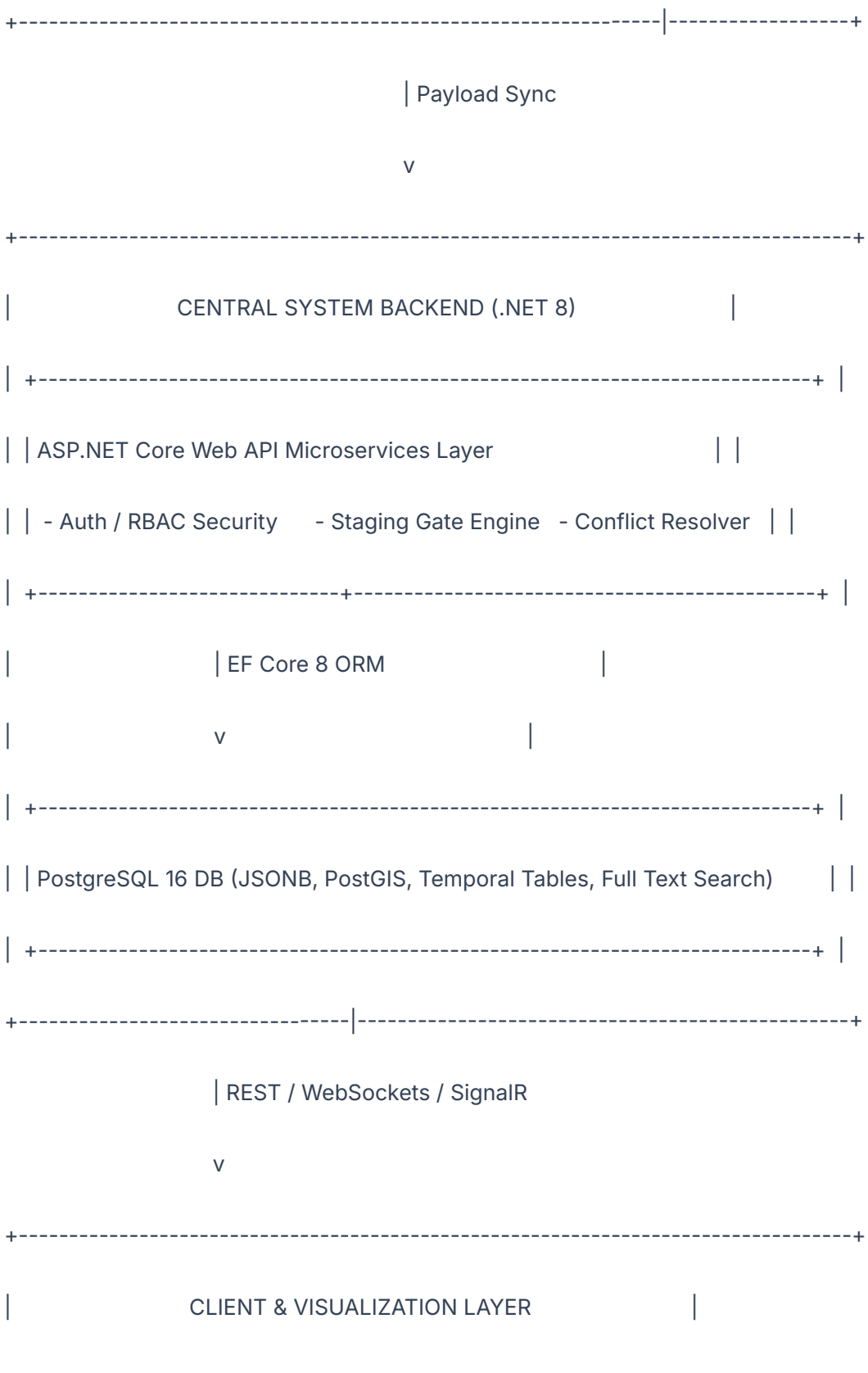
- **Eliminate Manual Data Entry:** Automated bi-directional parameter synchronization between native Revit/Forma models and structured relational database repositories.
- **Controlled Staging Gate (Option A Review):** High-integrity isolation layer preventing unverified field or project-management parameter alterations from corrupting central design models without explicit BIM Lead authorization.
- **Deterministic Parameter Persistence:** Multi-level element mapping schema enforcing strict GUID, `UniqueId`, and spatial indexing across multi-model federations.
- **Zero-Desktop-Footprint Automation:** Headless batch execution of model parameter updates using Autodesk Platform Services (APS) Design Automation API for Revit.
- **Real-time Spatial & Attribute Analytics:** Contextualized web visualizations utilizing the APS Viewer API, providing heatmapping, workset collision detection, and field-ready mobile barcode/QR interaction.

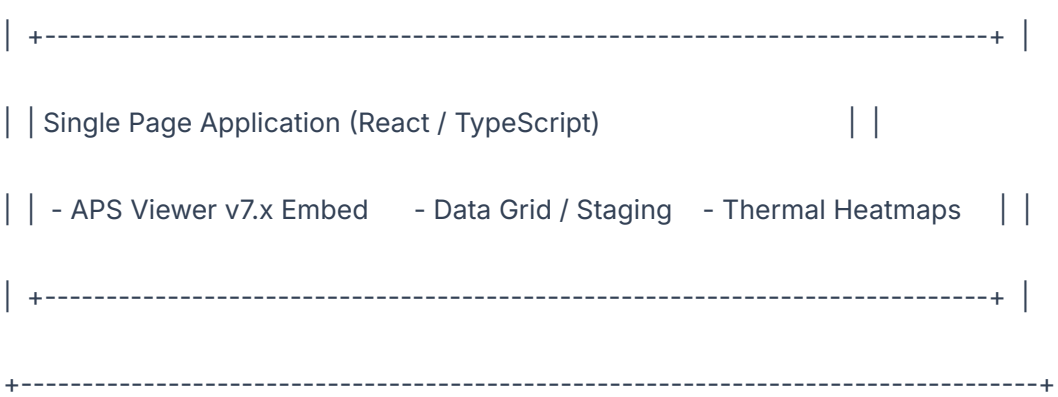
---

2. High-Level System Architecture

The ecosystem connects desktop design tools, cloud CAD repositories, headless automation engines, scalable microservice web backends, and browser-based 3D visualization clients.+----------+


AUTHORING LAYER
Autodesk Revit (Cloud WS)
Autodesk Forma (Site/Context)
Sync / Event Hooks
API / Embed
CLOUD INFRASTRUCTURE (ACC)
Autodesk Construction Cloud (ACC) Data Management / C4R Hub
Webhooks / SDK
APS & HEADLESS EXECUTION PIPELINE
APS Webhooks & Web API
APS Design Automation (Revit)


| Payload Sync
v
CENTRAL SYSTEM BACKEND (.NET 8)
ASP.NET Core Web API Microservices Layer
- Auth / RBAC Security
- Staging Gate Engine
- Conflict Resolver
| EF Core 8 ORM
v
PostgreSQL 16 DB (JSONB, PostGIS, Temporal Tables, Full Text Search)
| REST / WebSockets / SignalR
v
CLIENT & VISUALIZATION LAYER



System Components & Data Boundaries

1. **Authoring Layer:** Native BIM authoring inside Autodesk Revit (Cloud Workshared models in ACC) and conceptual context models in Autodesk Forma.
2. **APS Cloud Pipeline:** Autodesk Platform Services endpoints handle Model Derivative translation, Webhooks notification for model changes, and Design Automation for executing C# code headlessly against cloud-hosted `.rvt` files.
3. **Central Backend Core:** Built on .NET 8, providing secure RESTful API endpoints, background execution worker services via Quartz.NET, Object-Relational Mapping (ORM) via Entity Framework Core 8, and real-time client communication via SignalR.
4. **Data Store:** Enterprise PostgreSQL 16 instance configured with the PostGIS extension for geometric querying, native JSONB for dynamic parameter payloads, and System-Versioned Temporal Tables for complete audit compliance.
5. **Interactive Web Client:** Modern Web app built with React, TypeScript, and Tailwind CSS, embedding the two-way interactive APS Viewer v7.x for model-driven data editing and analytics.

---

3. Database Evaluation & Recommendation

A critical architectural decision is selecting the central repository for Master Equipment List data. The trade-offs between an off-the-shelf low-code solution (Smartsheet) and a custom-built relational database infrastructure (PostgreSQL + .NET 8) were evaluated.

Evaluation Criteria	Smartsheet / Low-Code Integration	Custom Database Platform (PostgreSQL + .NET 8)
Data Integrity & Constraints	Low. Poor foreign key enforcement; subject to user cell-override errors.	**Absolute.** Strict schema validation, foreign keys, constraints, and ACID transactions.
API Throughput & Rate Limits	High restriction. Tight API rate limits (e.g., 300 requests/min); unsuitable for bulk parameter syncs.	**Unrestricted.** High-throughput concurrent execution limited only by hosted infrastructure.
CAD/BIM Unique ID Indexing	Weak. Relies on text columns; no native composite keying or vector spatial indexing.	**Native.** Full support for UUID, text, dynamic JSONB indexing, and PostGIS spatial operators.
Auditability & Versioning	basic cell history; easily lost on row deletion or sheet restructuring.	**Enterprise.** System-versioned temporal tables track every parameter revision, user, and batch sync ID.
Granular Field Permissions	Row/Column level only; lacks dynamic approval-state security logic.	**Fine-Grained RBAC.** Role-Based and Attribute-Based Access Control down to individual parameter fields.
Cost at Scale	Exponential per-user licensing costs as non-BIM stakeholders increase.	**Fixed Infrastructure.** Scales linearly with cloud compute/storage; unlimited read/write users.

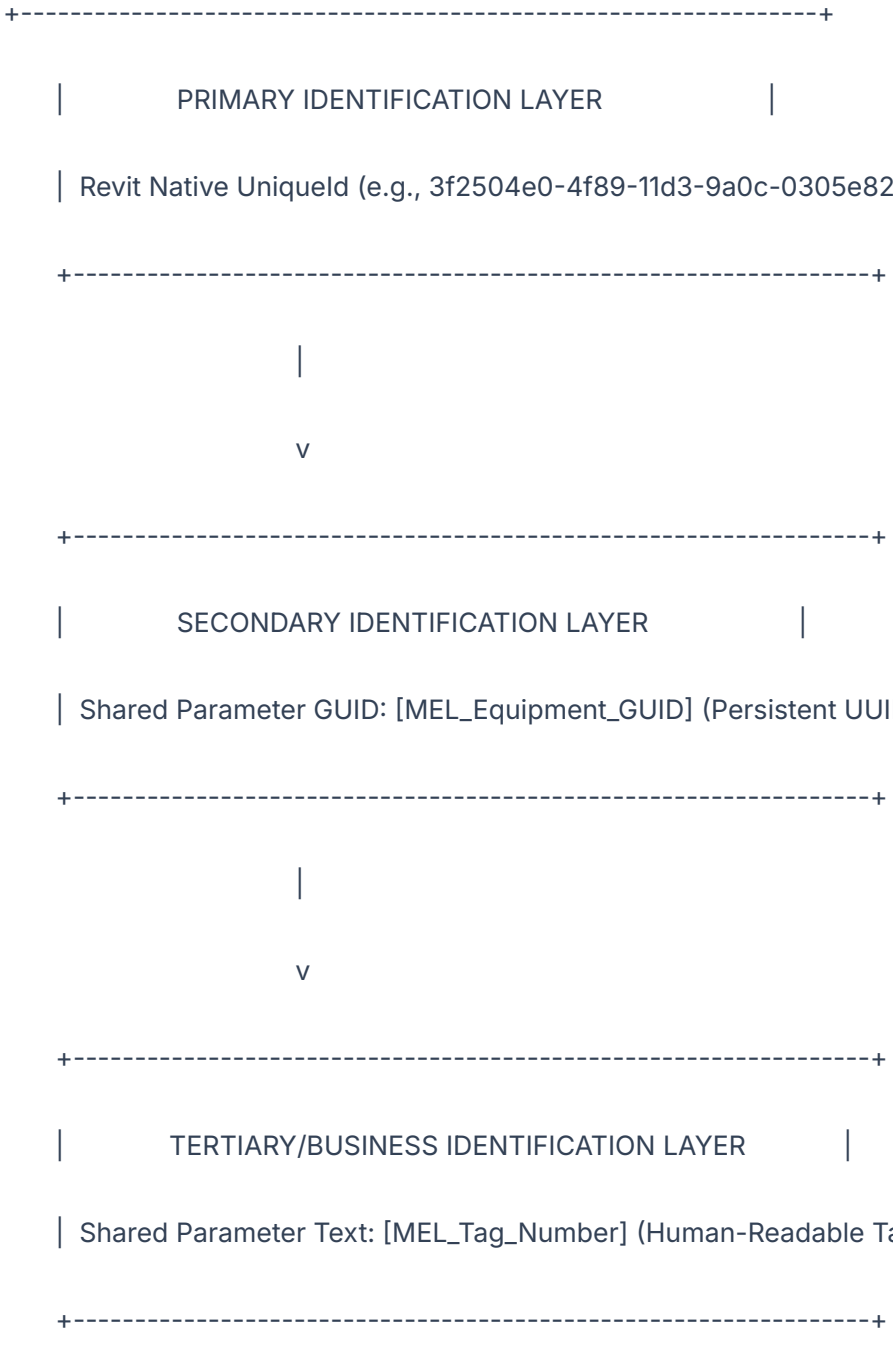
Architectural Recommendation

Deploy a Custom PostgreSQL + .NET 8 Backend. While Smartsheet offers rapid initial setup for simple lists, it fails under the structural, security, and throughput requirements of a federated multi-model BIM execution environment. The custom database architecture is required to reliably enforce the BIM Lead Approval Staging Gate, manage complex parameter mapping, and prevent sync conflicts.

---

4. Persistent Identifiers & Shared Parameter Strategy

To guarantee zero parameter loss or misallocation across complex multi-model BIM federations, the system implements a multi-layered identification and unit normalization architecture.



Shared Parameter Binding Matrix

Every Revit family destined for the MEL system must incorporate a standardized Shared Parameter File containing mandatory parameters with immutable GUID definitions:# NURBS / Shared Parameter File Definition

```
*PARAM  GUID                                  NAME                  DATATYPE      DATACATEGORY  GROUP  VISIBLE

PARAM   8f4a2110-14c1-4d92-a110-312984bc1001  MEL_Equipment_GUID    TEXT          -1            1      1

PARAM   1b2c3d4e-5f6a-7b8c-9d0e-1f2a3b4c5d6e  MEL_Tag_Number        TEXT          -1            1      1

PARAM   9d8c7b6a-5f4e-3d2c-1b0a-fe9d8c7b6a5f  MEL_Approval_Status   TEXT          -1            1      1

PARAM   3a2b1c0d-9e8f-7a6b-5c4d-3e2f1a0b9c8d  MEL_Operating_Weight  WEIGHT        -1            1      1

PARAM   7f8e9d0c-1b2a-3c4d-5e6f-7a8b9c0d1e2f  MEL_Electrical_Load
ELECTRICAL_POWER -1       1      1
```

Unit Normalization Protocol

To resolve inconsistencies between imperial and metric models across global design teams, the system enforces a strict internal normalization protocol:

1. **Ingest Layer:** All values extracted from Revit models are converted from native internal units (e.g., Imperial Feet, Pounds) or display units into standard **SI Base Units** prior to database storage.
2. **Storage Layer:** Values stored in PostgreSQL floating-point numeric columns are exclusively stored in standard SI Units (e.g., Kilograms, Kilowatts, Liters per Second).
3. **Egress/Display Layer:** The .NET API serializes unit types alongside raw values, allowing web clients or downstream target models to request dynamic conversions based on localized project unit schemas.

---

5. Headless APS Design Automation Engine

The core synchronization engine operates headlessly within Autodesk Platform Services (APS) Design Automation for Revit, running custom C# execution bundles against cloud-hosted models.using System;

```
using System.Collections.Generic;
using Autodesk.Revit.DB;
using Autodesk.Revit.ApplicationServices;
using DesignAutomationFramework;
using Newtonsoft.Json;
using System.IO;
namespace MEL.DesignAutomation
{
    [Autodesk.Revit.Attributes.Regeneration(Autodesk.Revit.Attributes.RegenerationOption.Manual)]
    [Autodesk.Revit.Attributes.Transaction(Autodesk.Revit.Attributes.TransactionMode.Manual)]
    public class SyncApp : IExternalDBApplication
    {
        public ExternalDBApplicationResult OnStartup(ControlledApplication application)
        {
            DesignAutomationBridge.DesignAutomationReadyEvent +=
HandleDesignAutomationReadyEvent;
            return ExternalDBApplicationResult.Succeeded;
        }
        public ExternalDBApplicationResult OnShutdown(ControlledApplication application)
        {
```

```
        return ExternalDBApplicationResult.Succeeded;
    }
    private void HandleDesignAutomationReadyEvent(object sender,
DesignAutomationReadyEventArgs e)
    {
        e.Succeeded = true;
        ExecuteSyncLogic(e.DesignAutomationData);
    }
    private void ExecuteSyncLogic(DesignAutomationData data)
    { loop:
        Document doc = data.RevitDoc;
        if (doc == null) return;
        string inputJsonPath = Path.Combine(Directory.GetCurrentDirectory(), "mel_updates.json");
        if (!File.Exists(inputJsonPath)) return;
        List<EquipmentParameterUpdate> updates =

JsonConvert.DeserializeObject<List<EquipmentParameterUpdate>>(File.ReadAllText(inputJsonPat
h));
        using (Transaction trans = new Transaction(doc, "MEL Parameter Automated Sync"))
        {
            trans.Start();
```

```
    // Advanced Failure Handling: Suppress non-critical warnings during batch updates
    FailureHandlingOptions failOptions = trans.GetFailureHandlingOptions();
    failOptions.SetFailuresPreprocessor(new WarningSwallower());
    trans.SetFailureHandlingOptions(failOptions);
    foreach (var update in updates)
    {
        Element targetElem = doc.GetElement(update.UniqueId);
        if (targetElem == null) continue;
        foreach (var paramEntry in update.Parameters)
        {
            Parameter param = targetElem.get_Parameter(new Guid(paramEntry.Key));
            if (param != null && !param.IsReadOnly)
            {
                ApplyParameterValue(param, paramEntry.Value);
            }
        }
    }
    trans.Commit();
}
// Central Model Worksharing Synchronization Options
```

```
        if (doc.IsWorkshared)
        {
            SynchronizeWithCentralOptions syncOptions = new SynchronizeWithCentralOptions();
            RelinquishOptions relinquishOptions = new RelinquishOptions(true);
            syncOptions.SetRelinquishOptions(relinquishOptions);
            syncOptions.SaveLocalAfter = false;
            syncOptions.Comment = "Automated Master Equipment List APS Parameter
Synchronization";
            TransactWithCentralOptions transactOptions = new TransactWithCentralOptions();

            try
            {
                WorksharingUtils.SynchronizeWithCentral(doc, transactOptions, syncOptions);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Worksharing Synchronization failed: {ex.Message}");
                throw;
            }
        }
```

```
    else
    {
        SaveOptions saveOpt = new SaveOptions();
        doc.Save(saveOpt);
    }
}
private void ApplyParameterValue(Parameter param, string value)
{
    switch (param.StorageType)
    {
        case StorageType.String:
            param.Set(value);
            break;
        case StorageType.Double:
            if (double.TryParse(value, out double dVal)) param.Set(dVal);
            break;
        case StorageType.Integer:
            if (int.TryParse(value, out int iVal)) param.Set(iVal);
            break;
```

```
        }
    }
}
public class WarningSwallower : IFailuresPreprocessor
{
    public FailureProcessingResult PreprocessFailures(FailuresAccessor failuresAccessor)
    {
        IList<FailureMessageAccessor> failList = failuresAccessor.GetFailureMessages();
        foreach (FailureMessageAccessor failure in failList)
        {
            if (failure.GetSeverity() == FailureSeverity.Warning)
            {
                failuresAccessor.DeleteWarning(failure);
            }
        }
        return FailureProcessingResult.Continue;
    }
}
public class EquipmentParameterUpdate
{
```

```
        public string UniqueId { get; set; }

        public Dictionary<string, string> Parameters { get; set; }

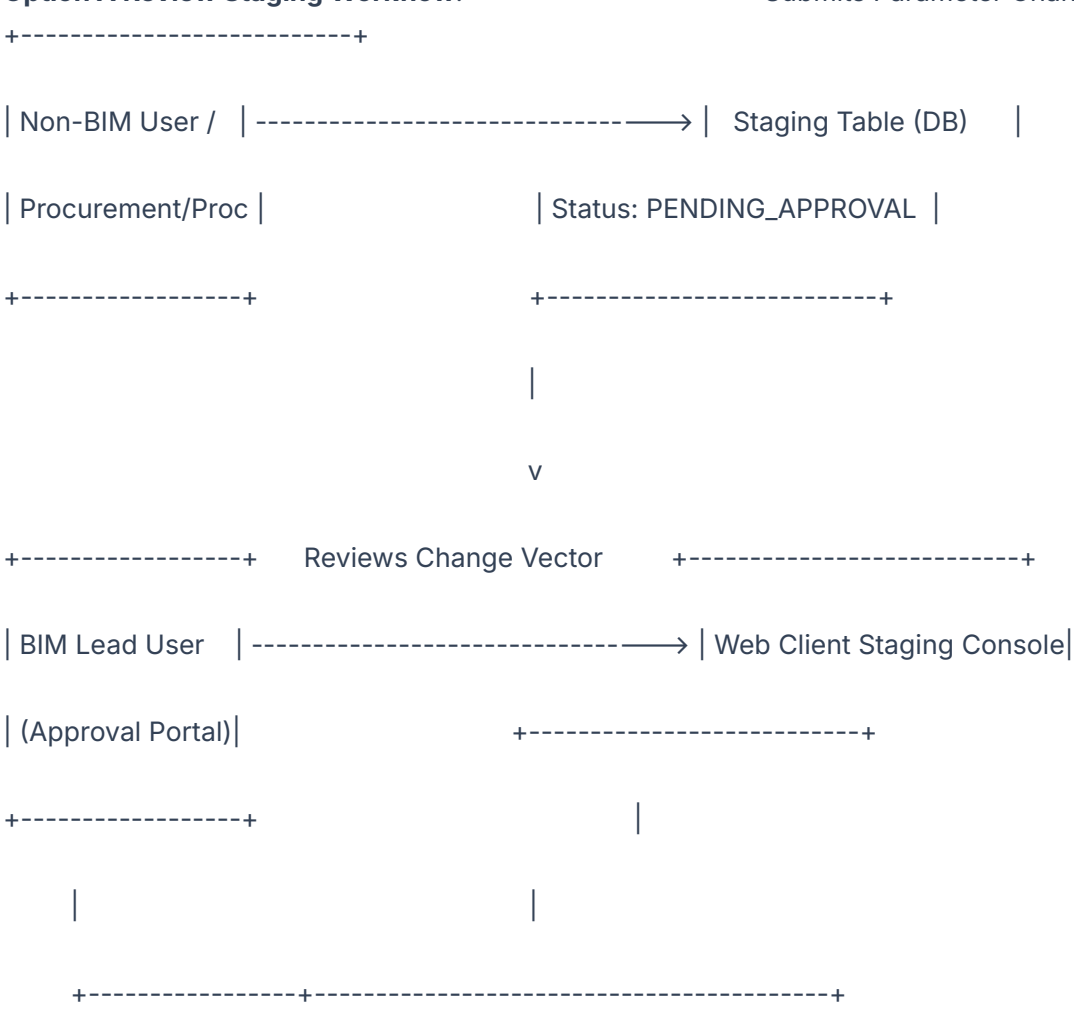
    }

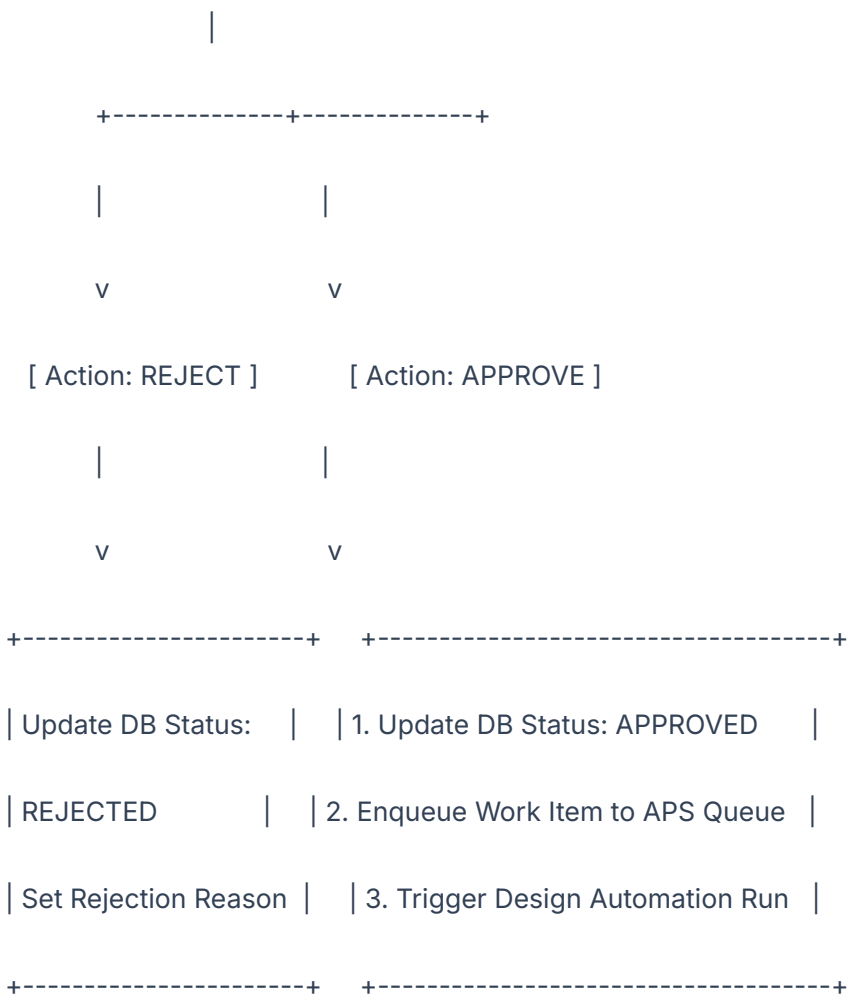
}
```

---

6. Central Web Backend & BIM Lead Approval Staging Gate

Directly writing unverified parameter changes from external procurement, electrical, or mechanical tools into native BIM models introduces unacceptable design risk. The system mitigates this via an **Option A Review Staging Workflow**.





EF Core Enterprise Entity Model Definition

```
using System;

using System.Collections.Generic;

using System.ComponentModel.DataAnnotations;

using System.ComponentModel.DataAnnotations.Schema;

namespace MEL.Core.Entities

{

    public enum ApprovalStatus
```

```
{
    Pending = 0,
    Approved = 1,
    Rejected = 2,
    CommittedToModel = 3
}
[Table("equipment_master")]
public class EquipmentMaster
{
    [Key]
    public Guid Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string UniqueId { get; set; } = string.Empty;
    [Required]
    [MaxLength(100)]
    public string ModelGuid { get; set; } = string.Empty;
    [Required]
    [MaxLength(100)]
    public string TagNumber { get; set; } = string.Empty;
```

```
    [Column(TypeName = "jsonb")]
    public string NormalizedParameters { get; set; } = "{}";
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    public ICollection<StagingParameterUpdate> PendingUpdates { get; set; }
        = new List<StagingParameterUpdate>();
}
[Table("staging_parameter_updates")]
public class StagingParameterUpdate
{
    [Key]
    public Guid Id { get; set; }
    public Guid EquipmentId { get; set; }
    [ForeignKey(nameof(EquipmentId))]
    public EquipmentMaster Equipment { get; set; } = null!;
    [Required]
    public string ParameterGuid { get; set; } = string.Empty;
    public string ParameterName { get; set; } = string.Empty;
    public string OldValue { get; set; } = string.Empty;
```

```
        public string ProposedValue { get; set; } = string.Empty;

        public ApprovalStatus Status { get; set; } = ApprovalStatus.Pending;

        [Required]

        public string RequestedBy { get; set; } = string.Empty;

        public string? ReviewedBy { get; set; }

        public string? RejectionReason { get; set; }

        public DateTime RequestedAt { get; set; } = DateTime.UtcNow;

        public DateTime? ReviewedAt { get; set; }

    }

}
```

RESTful API Endpoints (ASP.NET Core 8 Controller)

```
p
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MEL.Core.Entities;
using System.Threading.Tasks;
using System;
using System.Linq;

namespace MEL.Api.Controllers
{
[ApiController]
[Route("api/v1/[controller]")]
public class EquipmentStagingController : ControllerBase
{
private readonly MelDbContext _dbContext;    public EquipmentStagingController(MelDbContext
dbContext)

    {
```

```
    _dbContext = dbContext;
}
[HttpPost("stage-update")]
public async Task<IActionResult> StageParameterUpdate([FromBody] StageUpdateDto dto)
{
    var equipment = await _dbContext.EquipmentMaster
        .FirstOrDefaultAsync(e => e.UniqueId == dto.UniqueId);
    if (equipment == null) return NotFound("Equipment element not found.");
    var stagingEntry = new StagingParameterUpdate
    {
        Id = Guid.NewGuid(),
        EquipmentId = equipment.Id,
        ParameterGuid = dto.ParameterGuid,
        ParameterName = dto.ParameterName,
        OldValue = dto.CurrentValue,
        ProposedValue = dto.ProposedValue,
        Status = ApprovalStatus.Pending,
        RequestedBy = User.Identity?.Name ?? "External System",
        RequestedAt = DateTime.UtcNow
```

```
    };
    _dbContext.StagingParameterUpdates.Add(stagingEntry);
    await _dbContext.SaveChangesAsync();
    return Ok(new { StagingId = stagingEntry.Id, Status = "Pending Approval" });
}
[HttpPost("approve/{stagingId}")]
public async Task<IActionResult> ApproveStagedUpdate(Guid stagingId)
{
    var update = await _dbContext.StagingParameterUpdates
        .Include(s => s.Equipment)
        .FirstOrDefaultAsync(s => s.Id == stagingId);
    if (update == null) return NotFound();
    if (update.Status != ApprovalStatus.Pending) return BadRequest("Update is not pending.");
    update.Status = ApprovalStatus.Approved;
    update.ReviewedBy = User.Identity?.Name ?? "BIM Lead";
    update.ReviewedAt = DateTime.UtcNow;
    await _dbContext.SaveChangesAsync();
    // Trigger Background Task for Design Automation Execution
    // _queue.EnqueueApsSyncJob(update.Equipment.ModelGuid, update);
    return Ok(new { Message = "Change approved. Design Automation job queued." });
```

```
    }

}

public record StageUpdateDto(string UniqueId, string ParameterGuid, string ParameterName,
string CurrentValue, string ProposedValue)
;
}
```

---

7. Interactive Web Client & 3D APS Viewer Integration

The front-end integrates the Autodesk Platform Services (APS) Viewer v7.x to establish dynamic, bi-directional interaction between tabular data and 3D visual geometry, within an optional Autodesk Forma site context.

```typescript
import React, { useEffect, useRef, useState } from 'react';

declare const Autodesk: any;

interface ViewerProps {
  urn: string;
  token: string;
  onElementSelected?: (dbId: number, uniqueId: string) => void;
}

export const MelApsViewer: React.FC<ViewerProps> = ({ urn, token, onElementSelected }) => {
  const viewerContainer = useRef<HTMLDivElement>(null);
  const viewerRef = useRef<any>(null);
```

```
useEffect(() => {
  const options = {
    env: 'AutodeskProduction2',
    api: 'streamingV2',
    getAccessToken: (onTokenReady: (token: string, expires: number) => void) => {
      onTokenReady(token, 3600);
    }
  };
  Autodesk.Viewing.Initializer(options, () => {
    if (!viewerContainer.current) return;
    const viewer = new Autodesk.Viewing.GuiViewer3D(viewerContainer.current);
    viewer.start();
    viewerRef.current = viewer;
    const documentId = 'urn:' + urn;
    Autodesk.Viewing.Document.load(documentId, (doc: any) => {
      const defaultModel = doc.getRoot().getDefaultGeometry();
      viewer.loadDocumentNode(doc, defaultModel);
    });
    // Selection Event Mapping
```

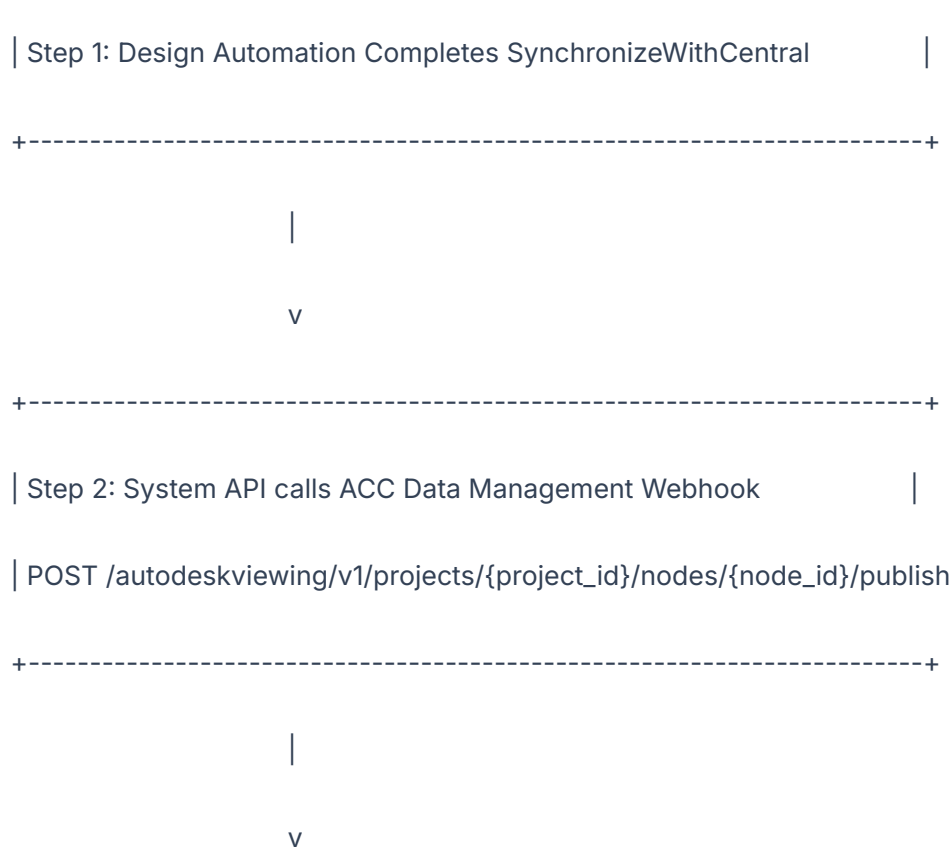
```
    viewer.addEventListener(Autodesk.Viewing.SELECTION_CHANGED_EVENT, (event: any) => {
      const dbIdArray = viewer.getSelection();
      if (dbIdArray.length === 1) {
        const dbId = dbIdArray[0];
        viewer.getProperties(dbId, (propData: any) => {
          const externalId = propData.externalId; // Maps directly to Revit UniqueId
          if (onElementSelected) {
            onElementSelected(dbId, externalId);
          }
        });
      }
    });
  });
  return () => {
    if (viewerRef.current) {
      viewerRef.current.finish();
    }
  };
}, [urn]);
return (
```

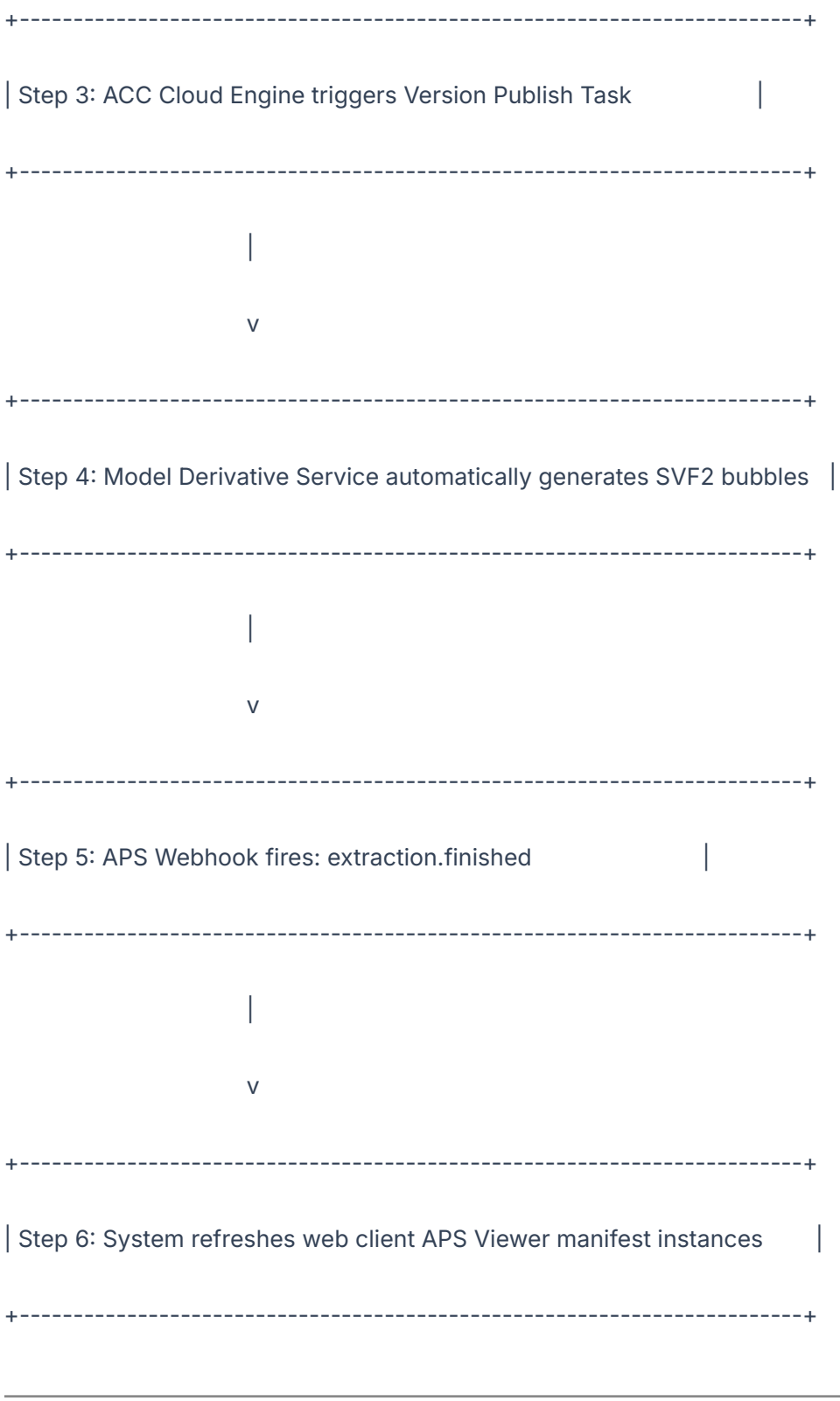
```
    <div className="relative w-full h-[600px] border border-slate-200 rounded-lg overflow-hidden">
      <div ref={viewerContainer} className="w-full h-full" />
    </div>
  );
};
```

---

8. Automated Post-Sync ACC Model Publishing Workflow

To ensure non-desk users observe up-to-date visualization states following an automated parameter update, the system triggers a post-sync publish sequence via ACC Webhooks.




Step 3: ACC Cloud Engine triggers Version Publish Task
Step 4: Model Derivative Service automatically generates SVF2 bubbles
Step 5: APS Webhook fires: extraction.finished
Step 6: System refreshes web client APS Viewer manifest instances

---

9. High-Value Stakeholder Features

Visual Parameter Heatmaps

The Web UI dynamically recolors model elements in the 3D Viewer based on real-time data status (e.g., approval state, lead-time risk, or electrical capacity limits) using `viewer.setThemingColor()`.t

```
// Heatmap Implementation: Color elements based on Approval State
public applyApprovalHeatmap(viewer: any, elementStatusMap: Map<number, string>) {
viewer.clearThemingColors();

const COLOR_APPROVED = new THREE.Vector4(0.1, 0.8, 0.2, 0.7); // Soft Green
const COLOR_PENDING = new THREE.Vector4(0.9, 0.6, 0.1, 0.7);  // Amber
const COLOR_REJECTED = new THREE.Vector4(0.9, 0.2, 0.2, 0.7); // Red

elementStatusMap.forEach((status, dbId) => {
if (status === 'Approved') {
viewer.setThemingColor(dbId, COLOR_APPROVED);
} else if (status === 'Pending') {
viewer.setThemingColor(dbId, COLOR_PENDING);
} else if (status === 'Rejected') {
viewer.setThemingColor(dbId, COLOR_REJECTED);
}
});
}
```

Workset Conflict & Lock Detection

Before executing an automated sync write-back, the .NET backend queries the model state to verify that target elements are not locked by active user sessions.

* **Checkout Checking:** Reads Revit `WorksharingUtils.GetCheckoutStatus()` via a preliminary quick-read container in APS.

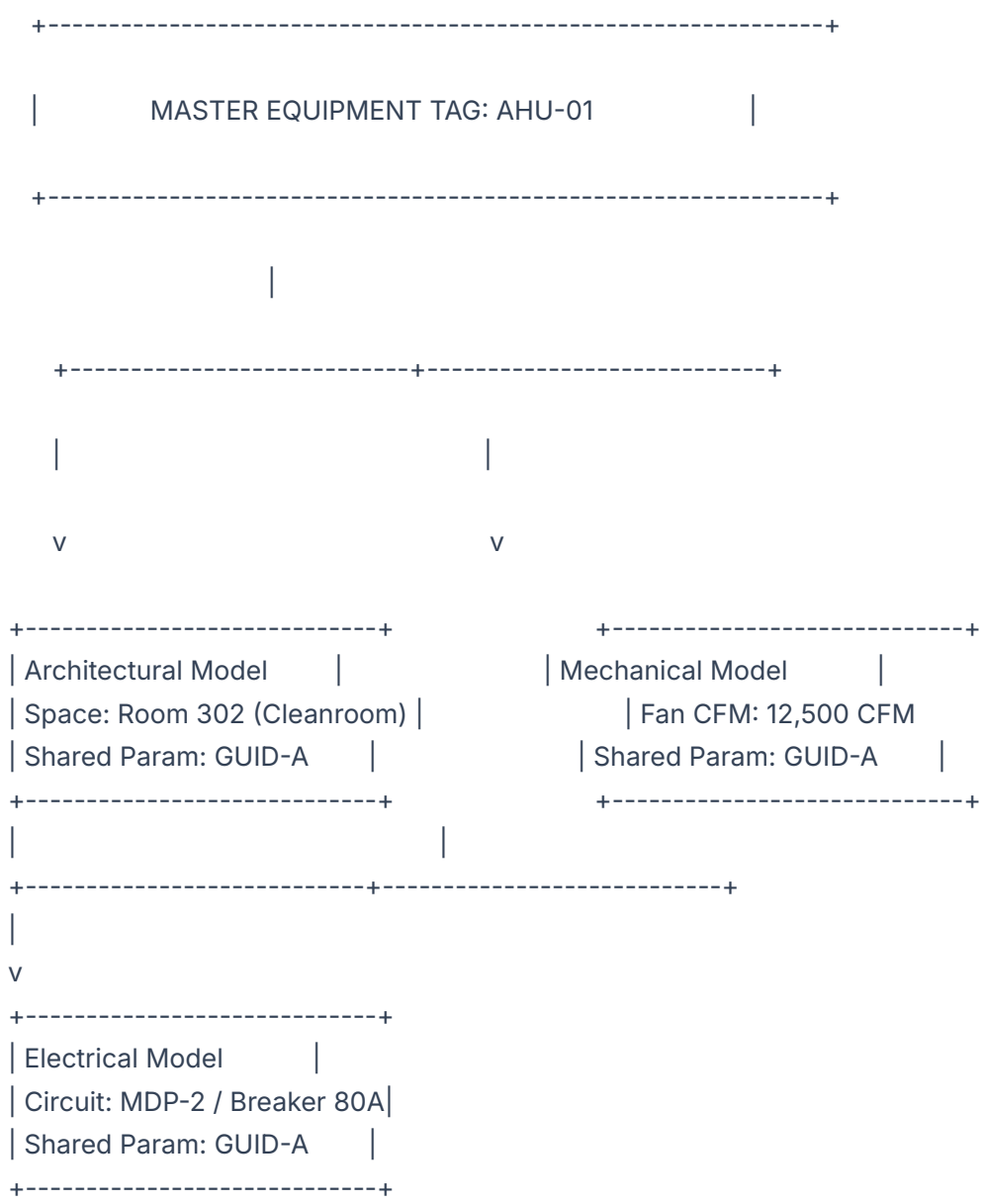
* **Soft-Locking System:** If a non-BIM user edits a record on the Web dashboard, the database places a 15-minute soft lock on that `UniqueId` to prevent simultaneous conflicting submissions.

Mobile Field Barcode / QR Scanning Integration

* **Generation:** When equipment records are committed, the server generates a cryptographically signed QR code encoding the specific `UniqueId` and direct web asset URL.

* **Field Action:** Field crews scan the QR tag via a mobile web device. The app opens the web dashboard, isolates the 3D model view using `viewer.select(dbId)` and `viewer.fitToView([dbId])`, and brings up parameter inspection and staging entry forms instantly.

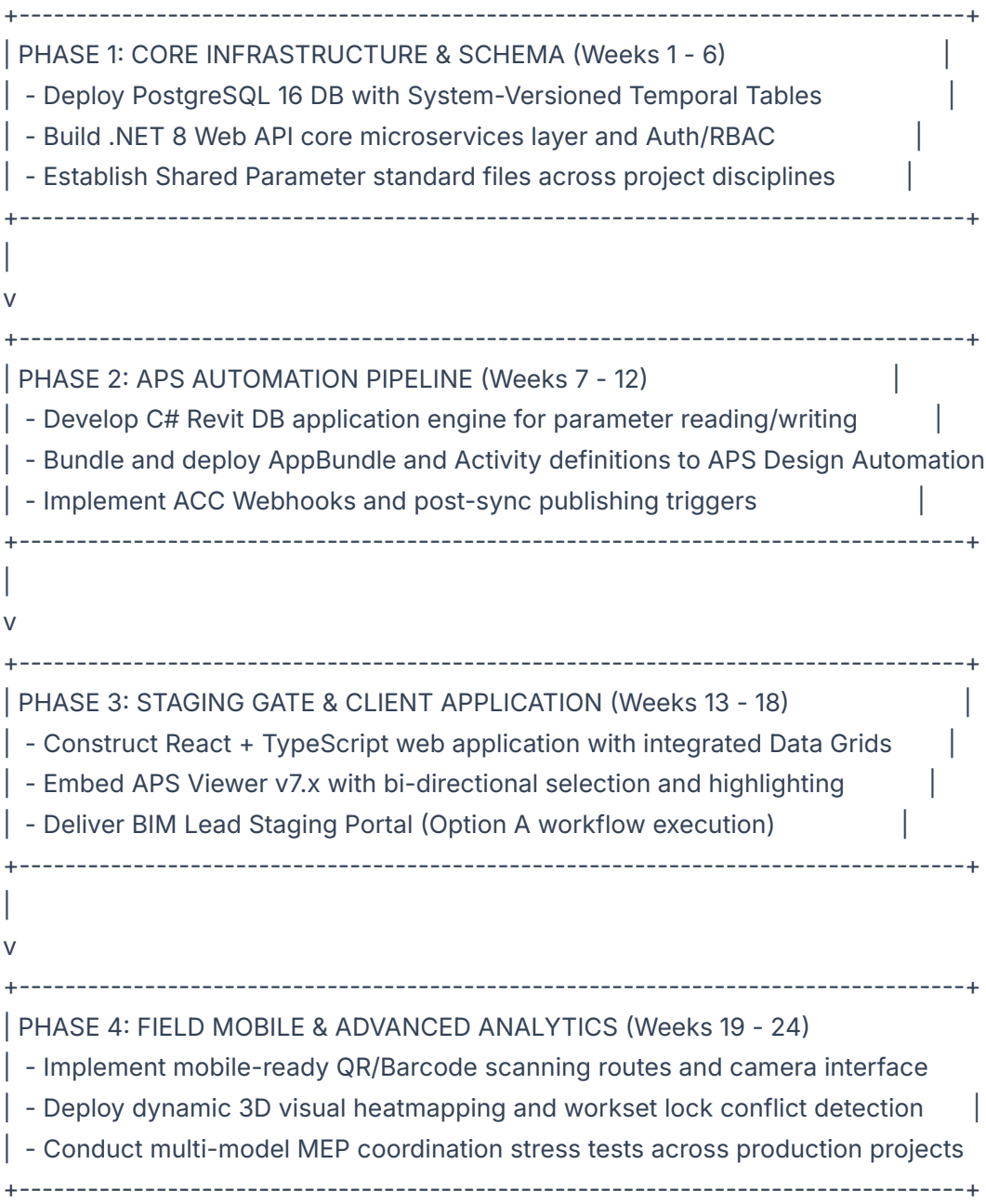
Multi-Model MEP Coordination Matrix



The system uses the persistent `MEL_Equipment_GUID` to link elements across disparate domain models (Architectural, Mechanical, Electrical, Structural). Changes to equipment capacity (e.g., motor horsepower) in the Mechanical model automatically alert the Electrical team via the Staging Gate to review circuit sizing changes.

---

10. Phased Implementation Roadmap



Key Performance Indicators (KPIs) for Implementation Success

1. **Synchronization Cycle Time:** Full model sync processing under 3 minutes via headless APS automation.

2. **Data Accuracy:** Zero unreviewed parameter overrides entering central design models.

3. **Cross-Discipline Alignment:** 100% correlation of shared equipment identifiers across Architectural, Mechanical, and Electrical federated models.